

BÁO CÁO ĐỀ TÀI

FASHION AGENT MULTIMODAL

Hệ thống tìm kiếm thời trang thông minh
dựa trên RAG và kiến trúc Agent ReAct

PATH 1: Text-to-Image Search

Sinh viên thực hiện: [Tên sinh viên]

MSSV: [MSSV]

Lớp: [Tên lớp]

Giảng viên hướng dẫn: [Tên giảng viên]

Học kỳ: [HK / Năm học]

[Tên thành phố], [Tháng/Năm]

Mục lục

1	Tổng quan hệ thống	2
1.1	Giới thiệu đề tài	2
1.2	Lộ trình phát triển hệ thống	2
1.3	So sánh RAG v1.0 và Agent v2.0	2
2	Tiền xử lý dữ liệu	4
2.1	Dataset và cấu trúc dữ liệu	4
2.2	Làm sạch dữ liệu	4
2.3	Sinh caption bằng LLM (Gemini)	5
2.4	Nhận diện màu sắc bằng LLM	7
3	Kiến trúc RAG v1.0 — Nền tảng tìm kiếm	9
3.1	Tổng quan cơ chế tìm kiếm	9
3.2	FashionCLIP Embedding — Semantic Token	10
3.2.1	Tại sao dùng FashionCLIP thay vì CLIP gốc?	10
3.3	BM25 — Tìm kiếm theo nhãn và màu sắc	12
3.4	RRF Fusion và Soft Relevance Scoring	14
4	Tích hợp PostgreSQL — Triển khai Server	16
4.1	Lý do chuyển từ CSV sang PostgreSQL	16
4.2	Lược đồ cơ sở dữ liệu	16
4.3	Lớp truy cập dữ liệu (Data Access Layer)	17
5	Agent v2.0 — PATH 1: Text-to-Image Search	21
5.1	Kiến trúc tổng thể Agent	21
5.2	Bước 1 — Phân loại Intent (Router)	22
5.3	Bước 2 — Clarification Gate	25
5.4	Bước 3 — Memory Agent (PostgreSQL)	26
5.5	Bước 4 — Vòng lặp ReAct	28
5.6	Bước 5 — LLM Synthesis và Memory Update	30

6 Ví dụ hoạt động end-to-end	33
6.1 Kịch bản: Tìm trang phục đi họp công ty	33
6.1.1 Lượt 1 — Truy vấn ban đầu	33
6.1.2 Lượt 2 — Follow-up với thông tin bổ sung	34
6.1.3 Kịch bản từ chối — Sản phẩm không có trong database	35
7 Giao diện và triển khai	36
7.1 Giao diện Gradio	36
7.2 Cấu hình triển khai trên server	37
8 Đánh giá và Kết luận	39
8.1 Chỉ số đánh giá (theo kế hoạch v2.0)	39
8.2 Điểm mạnh thiết kế	39
8.3 Rủi ro và hướng giải quyết	40
8.4 Kết luận	40

Chương 1

Tổng quan hệ thống

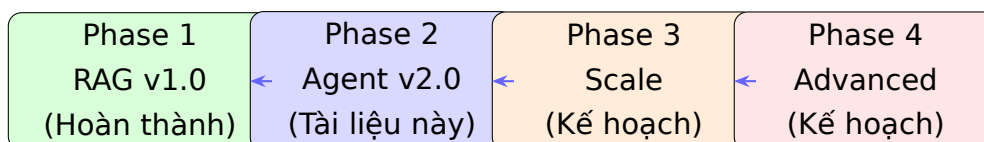
1.1 Giới thiệu đề tài

Đề tài xây dựng một hệ thống tìm kiếm thời trang thông minh dựa trên hai công nghệ nền tảng: **Retrieval-Augmented Generation (RAG)** và **AI Agent với vòng lặp ReAct**. Hệ thống được phát triển qua hai phiên bản:

- **RAG v1.0**: Pipeline cố định 9 bước, tìm kiếm sản phẩm thời trang bằng văn bản hoặc hình ảnh nhưng không có bộ nhớ ngữ cảnh.
- **Agent v2.0**: Tích hợp kiến trúc Agent chủ động, phân loại ý định, hỏi làm rõ khi thiếu thông tin, và lưu trữ dữ liệu trên **PostgreSQL**.

Tài liệu này trình bày **PATH 1 — Text-to-Image Search**: người dùng nhập truy vấn bằng văn bản tự nhiên (tiếng Việt hoặc tiếng Anh) và hệ thống trả về các sản phẩm thời trang phù hợp kèm tư vấn.

1.2 Lộ trình phát triển hệ thống



1.3 So sánh RAG v1.0 và Agent v2.0

Tiêu chí	RAG v1.0	Agent v2.0
Xử lý truy vấn	Pipeline cứng 9 bước	ReAct loop tự quyết định số bước
Truy vấn mơ hồ	Tìm kiếm với thông tin thiếu	Hỏi lại người dùng để làm rõ
Ngữ cảnh	Không nhớ giữa các lượt	MemoryAgent lưu sở thích phiên
Lưu trữ	CSV cục bộ	PostgreSQL (triển khai server)
Từ chối lịch sự	Không có	Phát hiện out-of-scope và từ chối
Kết quả	Danh sách sản phẩm	Sản phẩm + lý luận + gợi ý phối đồ

Chương 2

Tiền xử lý dữ liệu

2.1 Dataset và cấu trúc dữ liệu

Dữ liệu được lấy từ tập dataset *Clothing Dataset Full* (Kaggle), bao gồm hình ảnh thời trang cùng tệp `images.csv` với các cột sau:

Cột	Kiểu	Mô tả
image	string	ID ảnh (tên file không có phần mở rộng)
label	string	Nhãn danh mục: T-Shirt, Dress, Pants, ...
color	string	Màu sắc chủ đạo của sản phẩm
caption	string	Mô tả ngắn sinh bởi LLM (30–40 từ)

2.2 Làm sạch dữ liệu

Phương pháp: Lọc nhãn nhiễu và kiểm tra toàn vẹn

Loại bỏ các nhãn không rõ ràng (Not sure, Skip, Other), kiểm tra ảnh bị thiếu file, và xóa cột thừa.

```
1 import pandas as pd
2 import os
3 from PIL import Image
4
5 BASE_DIR = "/content/drive/MyDrive/RAG-practice/my_fashion_data"
6 CSV_PATH = os.path.join(BASE_DIR, "images.csv")
7 IMAGES_DIR = os.path.join(BASE_DIR, "images_compressed")
8
9 df = pd.read_csv(CSV_PATH)
```

```

10 print(f"Tong so dong: {len(df)}")
11
12 # 1. Loc nhan nhieu (khong ro rang)
13 NOISE_LABELS = ['Not sure', 'Skip', 'Other']
14 df_clean = df[~df['label'].isin(NOISE_LABELS)].copy()
15 print(f"Sau khi loc: {len(df_clean)} dong sach")
16
17 # 2. Kiem tra anh bi thieu
18 def get_image_path(image_id: str) -> str:
19     return os.path.join(IMGES_DIR, f"{image_id}.jpg")
20
21 missing = [iid for iid in df_clean['image']]
22             if not os.path.exists(get_image_path(iid))]
23 print(f"So anh thieu file: {len(missing)}")
24
25 # 3. Xoa cot thua
26 df_clean = df_clean.drop(columns=['sender_id', 'kids'], errors='ignore')
27
28 # 4. Luu lai
29 df_clean.to_csv(CSV_PATH, index=False)
30 print(f"Da luu: {CSV_PATH}")

```

Listing 2.1: Làm sạch dữ liệu ban đầu — processing_data.ipynb

Giải thích tác vụ:

1. `pd.read_csv` — Nạp toàn bộ dữ liệu vào DataFrame.
2. Lọc nhiễu bằng `isin` — giữ lại chỉ các nhãn có ý nghĩa thời trang rõ ràng.
3. Kiểm tra tệp ảnh để tránh lỗi khi embedding.
4. Xóa cột không liên quan trước khi sinh caption.

2.3 Sinh caption bằng LLM (Gemini)

Đây là bước quan trọng để bổ sung **semantic token** cho mỗi sản phẩm. Thay vì chỉ có nhãn danh mục đơn giản (*T-Shirt*), mỗi ảnh được Gemini mô tả chi tiết về cấu trúc trang phục, chất liệu, và kiểu dáng — tạo ra văn bản giàu ngữ nghĩa cho việc tìm kiếm.

Phương pháp: Multimodal Caption Generation

Gemini nhận đầu vào là **ảnh thực tế** của sản phẩm, sau đó sinh ra mô tả 30–40 từ tập trung vào: (1) chất liệu/kết cấu, (2) silhouette/kiểu dáng, (3) chi tiết cấu tạo, (4) phong cách tổng thể. Không đề cập màu sắc hay tên thương hiệu để đảm bảo tính khái quát.

```

1 import google.generativeai as genai
2 from tqdm import tqdm
3 import time
4
5 genai.configure(api_key="YOUR_API_KEY")
6 model = genai.GenerativeModel('gemini-2.5-flash')
7
8 def generate_search_caption(image_path: str) -> str | None:
9     """
10     Sinh mô tả 30-40 từ tập trung vào đặc điểm cấu tạo trang phục.
11     Đầu vào: đường dẫn ảnh
12     Đầu ra: chuỗi văn bản giàu ngữ nghĩa (semantic token)
13     """
14     prompt = """
15     Write a 30-40 word fashion-centric morphological description.
16     Focus:
17     1. Fabric / Texture
18     2. Silhouette / Fit
19     3. Construction details (collars, seams, closures)
20     4. Overall aesthetic
21
22     Rules:
23     - Use professional fashion vocabulary
24     - NO colors, NO brand names, NO generic category names
25     - Single paragraph only
26     """
27     try:
28         img = Image.open(image_path)
29         response = model.generate_content([prompt, img])
30         caption = (response.text.strip()
31                    .replace("\n", " ")
32                    .replace("*", ""))
33         return caption
34     except Exception as e:
35         print(f"Lỗi xử lý ảnh {image_path}: {e}")
36         return None
37
38 # Xử lý từng ảnh, lưu định kỳ
39 if "caption" not in df_clean.columns:

```



```

40     df_clean["caption"] = None
41
42 missing_captions = df_clean[df_clean["caption"].isna()]
43 print(f"Can xu ly: {len(missing_captions)} anh")
44
45 for idx, row in tqdm(missing_captions.iterrows(),
46                     total=len(missing_captions)):
47     img_path = get_image_path(row["image"])
48     if not os.path.exists(img_path):
49         continue
50     caption = generate_search_caption(img_path)
51     if caption:
52         df_clean.loc[idx, "caption"] = caption
53     if idx % 50 == 0:          # Lưu định kỳ
54         df_clean.to_csv(CSV_PATH, index=False)
55     time.sleep(1)             # Tránh rate limit

```

Listing 2.2: Sinh caption thời trang bằng Gemini — processing_data.ipynb

Giải thích tác vụ:

- **Multimodal input:** Gemini nhận đồng thời prompt văn bản và ảnh sản phẩm → sinh mô tả ngữ nghĩa.
- **Semantic token:** Caption kết quả là *semantic token* phong phú hơn nhãn ngắn, ví dụ: “relaxed-fit woven cotton pullover with ribbed crew neckline and hemline, drop shoulders, minimal seaming” — cung cấp thông tin chi tiết cho FashionCLIP embedding.
- **Rate limiting:** `time.sleep(1)` tránh vượt giới hạn API.
- **Checkpoint:** Lưu mỗi 50 ảnh phòng khi bị ngắt kết nối.

2.4 Nhận diện màu sắc bằng LLM

```

1 def detect_item_color(image_path: str, label: str) -> str | None:
2     """
3     Nhận diện màu sắc chủ đạo của sản phẩm thời trang.
4     Trả về tên màu cụ thể (vd: 'Olive Green' thay vì 'Green').
5     """
6     try:
7         img = Image.open(image_path)
8         prompt = f"""
9         Identify the primary color of this {label}.
10        Rules:
11        1. Return only the color name(s) – max 3-4 words.

```

```
12     2. Be specific: 'Charcoal Grey' not 'Grey'.
13     3. Pattern: mention base + pattern type
14         (e.g., 'White with Black stripes').
15     """
16     response = model.generate_content([prompt, img])
17     return response.text.strip().replace('*', '')
18 except Exception as e:
19     print(f"Loi nhan dien mau: {e}")
20     return None
```

Listing 2.3: Phát hiện màu sắc chủ đạo bằng Gemini

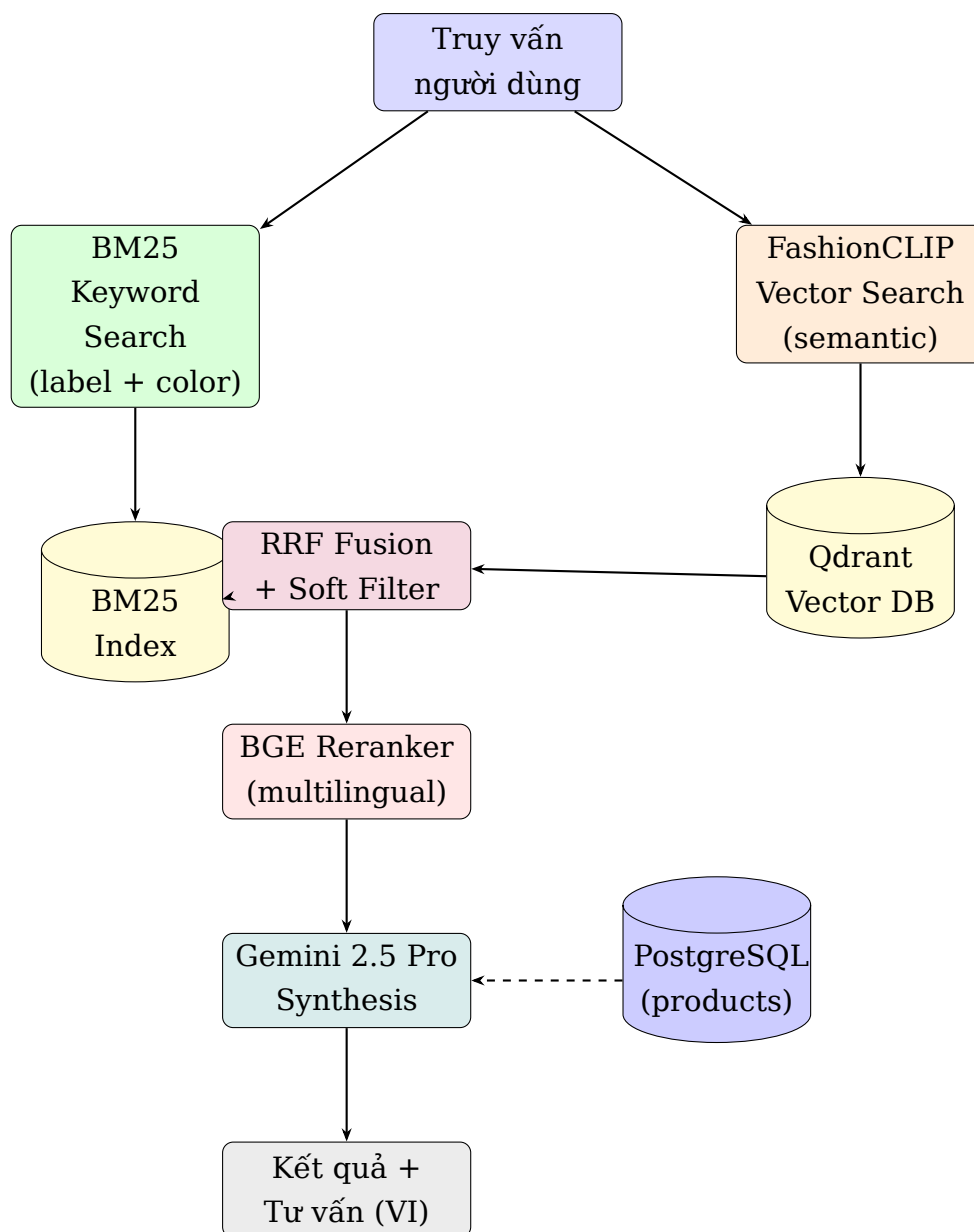
Giải thích: Màu sắc được phát hiện cụ thể và chi tiết (“Olive Green” thay vì chỉ “Green”), phục vụ cho bộ lọc mềm (*soft filter*) trong pipeline tìm kiếm.

Chương 3

Kiến trúc RAG v1.0 — Nền tảng tìm kiếm

3.1 Tổng quan cơ chế tìm kiếm

RAG (Retrieval-Augmented Generation) trong hệ thống này kết hợp hai phương pháp tìm kiếm bổ trợ nhau:



3.2 FashionCLIP Embedding — Semantic Token

3.2.1 Tại sao dùng FashionCLIP thay vì CLIP gốc?

FashionCLIP 2.0 (patrickjohncyh/fashion-clip) là mô hình ViT-B/32 được fine-tune trên 800.000 sản phẩm Farfetch, đạt Recall@1 tăng $\approx 40\%$ so với CLIP gốc trên dữ liệu thời trang. Mô hình cho ra vector 512 chiều đồng nhất giữa text và ảnh, cho phép so sánh ngữ nghĩa đa phương thức.

Phương pháp: Multimodal Embedding với FashionCLIP

Với mỗi sản phẩm, hai loại vector được tính:

1. **Text embedding**: từ caption (semantic token sinh bởi LLM) → bắt được ngữ nghĩa sâu.
2. **Image embedding**: từ file ảnh gốc → bắt được đặc trưng thị giác.

Truy vấn của người dùng cũng được embed thành vector text cùng không gian → so sánh cosine similarity.

```

1 from llama_index.core.embeddings import MultiModalEmbedding
2 from transformers import CLIPModel, CLIPProcessor
3 import torch
4 from typing import List
5
6 class FashionCLIPEmbedding(MultiModalEmbedding):
7     """
8     Custom embedding model dựa trên FashionCLIP 2.0.
9     Output: vector 512 chiều, chuẩn hóa L2.
10    """
11    MODEL_ID: str = 'patrickjohncyh/fashion-clip'
12
13    def __init__(self, model_name='patrickjohncyh/fashion-clip',
14                 embed_batch_size=32, **kwargs):
15        super().__init__(embed_batch_size=embed_batch_size,
16                         model_name=model_name, **kwargs)
17        self._device = 'cuda' if torch.cuda.is_available() else 'cpu'
18        self._model = CLIPModel.from_pretrained(model_name).to(self._device)
19        self._processor = CLIPProcessor.from_pretrained(model_name)
20        self._model.eval()
21
22    def _get_text_embedding(self, text: str) -> List[float]:
23        """Chuyen van ban thanh vector 512-d (dung cho query va caption)."""
24        inputs = self._processor(
25            text=[text], return_tensors='pt',
26            padding=True, truncation=True, max_length=77
27        ).to(self._device)
28        with torch.no_grad():
29            feat = self._model.get_text_features(**inputs)
30            feat = feat / feat.norm(dim=-1, keepdim=True) # chuẩn hóa L2
31        return feat[0].cpu().tolist()
32
33    def _get_image_embedding(self, img_file_path: str) -> List[float]:
34        """Chuyen anh thanh vector 512-d (dung cho Image Node)."""
35        image = Image.open(img_file_path).convert('RGB')
36        inputs = self._processor(images=image,

```

```

37         return_tensors='pt').to(self._device)
38     with torch.no_grad():
39         feat = self._model.get_image_features(**inputs)
40         feat = feat / feat.norm(dim=-1, keepdim=True)
41     return feat[0].cpu().tolist()
42
43     # Các phương thức khác: _get_text_embeddings, _get_image_embeddings,
44     # _get_query_embedding, và các phương thức async tương ứng.

```

Listing 3.1: Class FashionCLIPEmbedding — custom embedding model**Giải thích tác vụ:**

- CLIPProcessor: Tiền xử lý văn bản/ảnh theo đúng định dạng của mô hình.
- get_text_features / get_image_features: Lấy embedding đặc trưng từ encoder tương ứng.
- **Chuẩn hóa L2:** $\hat{v} = v / \|v\|_2$ đảm bảo dot product $\equiv$ cosine similarity, giúp tìm kiếm nhất quán.

3.3 BM25 — Tìm kiếm theo nhãn và màu sắc

BM25 (Best Match 25) là thuật toán xếp hạng từ khóa kinh điển. Trong hệ thống, BM25 được áp dụng trên nội dung ngắn "label. color." — ví dụ: "T-Shirt. Navy Blue." — cho phép tìm kiếm chính xác theo **tên danh mục** và **màu sắc cụ thể**.

Phương pháp: Hybrid Search — BM25 + FashionCLIP

- **BM25** mạnh ở: tìm chính xác tên category, màu sắc ("navy blue shirt").
- **FashionCLIP Vector** mạnh ở: hiểu ngữ nghĩa, đồng nghĩa, mô tả đặc trưng ("slim fit cotton casual").
- **RRF Fusion:** kết hợp kết quả từ cả hai với hệ số $k = 60$, bm25_weight=2.5.

```

1 from llama_index.core import Document
2 from llama_index.core.schema import ImageDocument, MetadataMode
3
4 def prepare_documents(df: pd.DataFrame, img_dir: str):
5     """
6     Tao Text Document (cho BM25 + Vector) và

```

```

7   Image Document (cho FashionCLIP Image Encoder).
8   """
9   text_docs, image_docs = [], []
10
11  for _, row in df.iterrows():
12      image_id = str(row["image"])
13      caption = str(row.get("caption", "")) # semantic token
14      label = str(row.get("label", "")) # category label
15      color = str(row.get("color", ""))
16
17      img_path = find_image_path(image_id, img_dir)
18      if img_path is None:
19          continue
20
21      metadata = {
22          "image_id": image_id, "image_path": img_path,
23          "category": label, "color": color,
24          "full_description": caption
25      }
26
27      # --- Text Document ---
28      # BM25 tìm kiếm trên nội dung ngắn: "label. color."
29      # FashionCLIP embed cả nội dung + metadata đầy đủ
30      bm25_content = f"{label}. {color}."
31      td = Document(text=bm25_content, metadata=metadata)
32      td.excluded_embed_metadata_keys = ["image_id", "image_path",
33                                         "category", "color"]
34      td.excluded_llm_metadata_keys = ["image_id", "image_path"]
35      text_docs.append(td)
36
37      # --- Image Document ---
38      # FashionCLIP Image Encoder xử lý ảnh thị giác
39      id_ = ImageDocument(image_path=img_path, metadata=metadata)
40      id_.excluded_embed_metadata_keys = ["image_id", "category",
41                                         "color", "full_description"]
42      image_docs.append(id_)
43
44  return text_docs, image_docs

```

Listing 3.2: Chuẩn bị Document cho BM25 và Vector Index**Giải thích tác vụ:**

- **BM25 content** (bm25_content): Chỉ dùng label + color — gọn, đặc hiệu, khớp tốt với truy vấn từ khóa.
- **LLM context** (full_description): caption đầy đủ được cung cấp cho

Gemini khi tổng hợp câu trả lời.

- `excluded_embed_metadata_keys`: Loại trừ metadata không cần thiết khỏi vector embedding để tránh nhiễu.

3.4 RRF Fusion và Soft Relevance Scoring

```

1 from typing import List, Dict
2 from llama_index.core.schema import NodeWithScore
3
4 def rrf_fusion(bm25_nodes: List[NodeWithScore],
5               vector_nodes: List[NodeWithScore],
6               k: int = 60,
7               bm25_weight: float = 2.5) -> List[NodeWithScore]:
8     """
9     Ket hop ket qua BM25 va Vector Search bang thuat toan
10    Reciprocal Rank Fusion (RRF).
11
12    Score = sum(1 / (k + rank)) voi trong so BM25 cao hon
13    vi keyword matching rat dac hieu voi fashion category.
14    """
15    fused = {}
16
17    for rank, node in enumerate(vector_nodes):
18        nid = node.node_id
19        if nid not in fused:
20            fused[nid] = {"node": node, "score": 0.0}
21            fused[nid]["score"] += 1.0 / (k + rank + 1)
22
23    for rank, node in enumerate(bm25_nodes):
24        nid = node.node_id
25        if nid not in fused:
26            fused[nid] = {"node": node, "score": 0.0}
27            # BM25 duoc nhan them trong so
28            fused[nid]["score"] += bm25_weight / (k + rank + 1)
29
30    sorted_items = sorted(fused.values(),
31                          key=lambda x: x["score"], reverse=True)
32    for item in sorted_items:
33        item["node"].score = item["score"]
34    return [item["node"] for item in sorted_items]
35
36
37 def calculate_relevance_score(node: NodeWithScore,
38                              filters: Dict) -> float:
39     """

```



```

40     Soft filter: tính điểm tu tương đồng (0.0-1.0) thay vì
41     hard filter 0/1. Giữ lại sản phẩm có relevance >= threshold.
42     """
43     if not filters:
44         return 1.0
45     actual = node.node if hasattr(node, 'node') else node
46     meta = actual.metadata
47     scores = []
48     if "color" in filters:
49         scores.append(calculate_color_similarity(
50             filters["color"], meta.get("color", "")))
51     if "category" in filters:
52         scores.append(calculate_category_similarity(
53             filters["category"], meta.get("category", "")))
54     return sum(scores) / len(scores) if scores else 1.0

```

Listing 3.3: Reciprocal Rank Fusion kết hợp BM25 và Vector results

Giải thích tác vụ:

- **RRF:** Tránh phụ thuộc vào điểm tuyệt đối của từng retriever (khác đơn vị). Mỗi kết quả được xếp hạng theo vị trí (rank), sau đó cộng điểm nghịch đảo.
- **bm25_weight = 2.5:** BM25 nhận tín hiệu mạnh hơn vì keyword match đặc hiệu hơn với tên category.
- **Soft filter:** calculate_relevance_score dùng RapidFuzz để so sánh màu sắc/category một cách mềm dẻo, không loại bỏ hoàn toàn kết quả không khớp chính xác.

Chương 4

Tích hợp PostgreSQL — Triển khai Server

4.1 Lý do chuyển từ CSV sang PostgreSQL

Lưu ý

CSV không phù hợp cho môi trường server vì: (1) không hỗ trợ truy cập đồng thời (concurrent access), (2) mất dữ liệu khi container khởi động lại, (3) không thể query có điều kiện hiệu quả. PostgreSQL giải quyết tất cả các vấn đề này, đồng thời cung cấp đầy đủ tính năng ACID.

4.2 Lược đồ cơ sở dữ liệu

```
1  -- Kích hoạt extension vector (pgvector) nếu cần lưu embedding
2  -- CREATE EXTENSION IF NOT EXISTS vector;
3
4  -- Bảng sản phẩm chính
5  CREATE TABLE fashion_products (
6      id          SERIAL PRIMARY KEY,
7      image_id    VARCHAR(100) UNIQUE NOT NULL,
8      label       VARCHAR(100) NOT NULL,    -- category: T-Shirt, Dress...
9      color       VARCHAR(200),             -- màu sắc chi tiết
10     caption     TEXT,                     -- semantic token sinh bởi LLM
11     gcs_path     TEXT,                   -- gs://bucket/path/to/image.jpg
12     created_at  TIMESTAMP DEFAULT NOW()
13 );
14
```

```

15 -- Index de tang toc query theo label va color
16 CREATE INDEX idx_products_label ON fashion_products(label);
17 CREATE INDEX idx_products_color ON fashion_products(color);
18 CREATE INDEX idx_products_label_color
19     ON fashion_products(label, color);
20
21 -- Bang session nguoi dung (luu preference)
22 CREATE TABLE user_sessions (
23     session_id VARCHAR(64) PRIMARY KEY,
24     liked_items JSONB DEFAULT '[]',      -- [{image_id, label, color}]
25     query_history JSONB DEFAULT '[]',    -- [{query, intent, ts}]
26     created_at TIMESTAMP DEFAULT NOW(),
27     updated_at TIMESTAMP DEFAULT NOW()
28 );
29
30 -- Bang feedback nguoi dung
31 CREATE TABLE agent_feedback (
32     id SERIAL PRIMARY KEY,
33     session_id VARCHAR(64),
34     query TEXT,
35     result_ids TEXT[],
36     is_relevant BOOLEAN,
37     comment TEXT,
38     created_at TIMESTAMP DEFAULT NOW()
39 );

```

Listing 4.1: Schema PostgreSQL cho hệ thống Fashion Agent

4.3 Lớp truy cập dữ liệu (Data Access Layer)

```

1 import psycopg2
2 import psycopg2.extras
3 import pandas as pd
4 from typing import List, Dict, Optional
5 import json
6
7 class FashionProductDB:
8     """
9     Lớp truy cập PostgreSQL cho dữ liệu sản phẩm thời trang.
10    Thay thế hoàn toàn việc đọc CSV cục bộ.
11    """
12
13    def __init__(self, dsn: str):
14        """
15        dsn: PostgreSQL connection string
16        Ví dụ: "postgresql://user:pass@localhost:5432/fashion_db"

```

```

17     Hoac lay tu bien moi truong: os.environ['DATABASE_URL']
18     """
19     self.dsn = dsn
20
21     def _connect(self):
22         return psycopg2.connect(self.dsn)
23
24     # — Nap du lieu san pham —————
25     def load_products(self, limit: int = 5000) -> pd.DataFrame:
26         """Nap san pham tu PostgreSQL thay the pd.read_csv()."""
27         sql = """
28             SELECT image_id, label, color, caption, gcs_path
29             FROM fashion_products
30             WHERE caption IS NOT NULL
31             ORDER BY id
32             LIMIT %s
33         """
34         with self._connect() as conn:
35             df = pd.read_sql(sql, conn, params=(limit,))
36             print(f"Da tai {len(df)} san pham tu PostgreSQL")
37             return df
38
39     # — Import CSV vao PostgreSQL (chay 1 lan) —————
40     def ingest_from_csv(self, csv_path: str,
41                         img_dir: str, max_items: int = 2000):
42         """Chuyen du lieu tu CSV sang PostgreSQL (idempotent)."""
43         # Kiem tra neu da co du lieu
44         with self._connect() as conn:
45             with conn.cursor() as cur:
46                 cur.execute("SELECT COUNT(*) FROM fashion_products")
47                 count = cur.fetchone()[0]
48             if count >= 100:
49                 print(f"PostgreSQL da co {count} san pham, bo qua ingest.")
50                 return
51
52         df = pd.read_csv(csv_path).head(max_items).fillna("")
53         insert_sql = """
54             INSERT INTO fashion_products
55                 (image_id, label, color, caption)
56             VALUES %s
57             ON CONFLICT (image_id) DO NOTHING
58         """
59         rows = []
60         for _, row in df.iterrows():
61             rows.append((str(row["image"]),
62                          str(row.get("label", "")),
63                          str(row.get("color", "")),

```

```

64         str(row.get("caption", ""))))
65
66     with self._connect() as conn:
67         with conn.cursor() as cur:
68             psycopg2.extras.execute_values(cur, insert_sql, rows,
69                                             page_size=200)
70         conn.commit()
71     print(f"Đã import {len(rows)} sản phẩm vào PostgreSQL")
72
73     # — Session management —————
74     def get_session(self, session_id: str) -> Dict:
75         sql = "SELECT * FROM user_sessions WHERE session_id = %s"
76         with self._connect() as conn:
77             with conn.cursor(
78                 cursor_factory=psycopg2.extras.DictCursor) as cur:
79                 cur.execute(sql, (session_id,))
80                 row = cur.fetchone()
81             return dict(row) if row else {}
82
83     def upsert_session(self, session_id: str, data: Dict):
84         sql = """
85             INSERT INTO user_sessions (session_id, liked_items, query_history)
86             VALUES (%s, %s, %s)
87             ON CONFLICT (session_id) DO UPDATE SET
88                 liked_items = EXCLUDED.liked_items,
89                 query_history = EXCLUDED.query_history,
90                 updated_at = NOW()
91         """
92         with self._connect() as conn:
93             with conn.cursor() as cur:
94                 cur.execute(sql, (
95                     session_id,
96                     json.dumps(data.get("liked_items", [])),
97                     json.dumps(data.get("query_history", []))
98                 ))
99             conn.commit()
100
101     # — Lưu feedback —————
102     def save_feedback(self, session_id: str, query: str,
103                      result_ids: List[str], is_relevant: bool,
104                      comment: str = ""):
105         sql = """
106             INSERT INTO agent_feedback
107             (session_id, query, result_ids, is_relevant, comment)
108             VALUES (%s, %s, %s, %s, %s)
109         """
110         with self._connect() as conn:

```

```

111         with conn.cursor() as cur:
112             cur.execute(sql, (session_id, query, result_ids,
113                             is_relevant, comment))
114         conn.commit()
115
116     # Khởi tạo kết nối
117     import os
118     db = FashionProductDB(
119         dsn=os.environ.get("DATABASE_URL",
120                             "postgresql://user:pass@localhost:5432/fashion_db")
121     )

```

Listing 4.2: PostgreSQL Data Access Layer với psycopg2

Giải thích tác vụ:

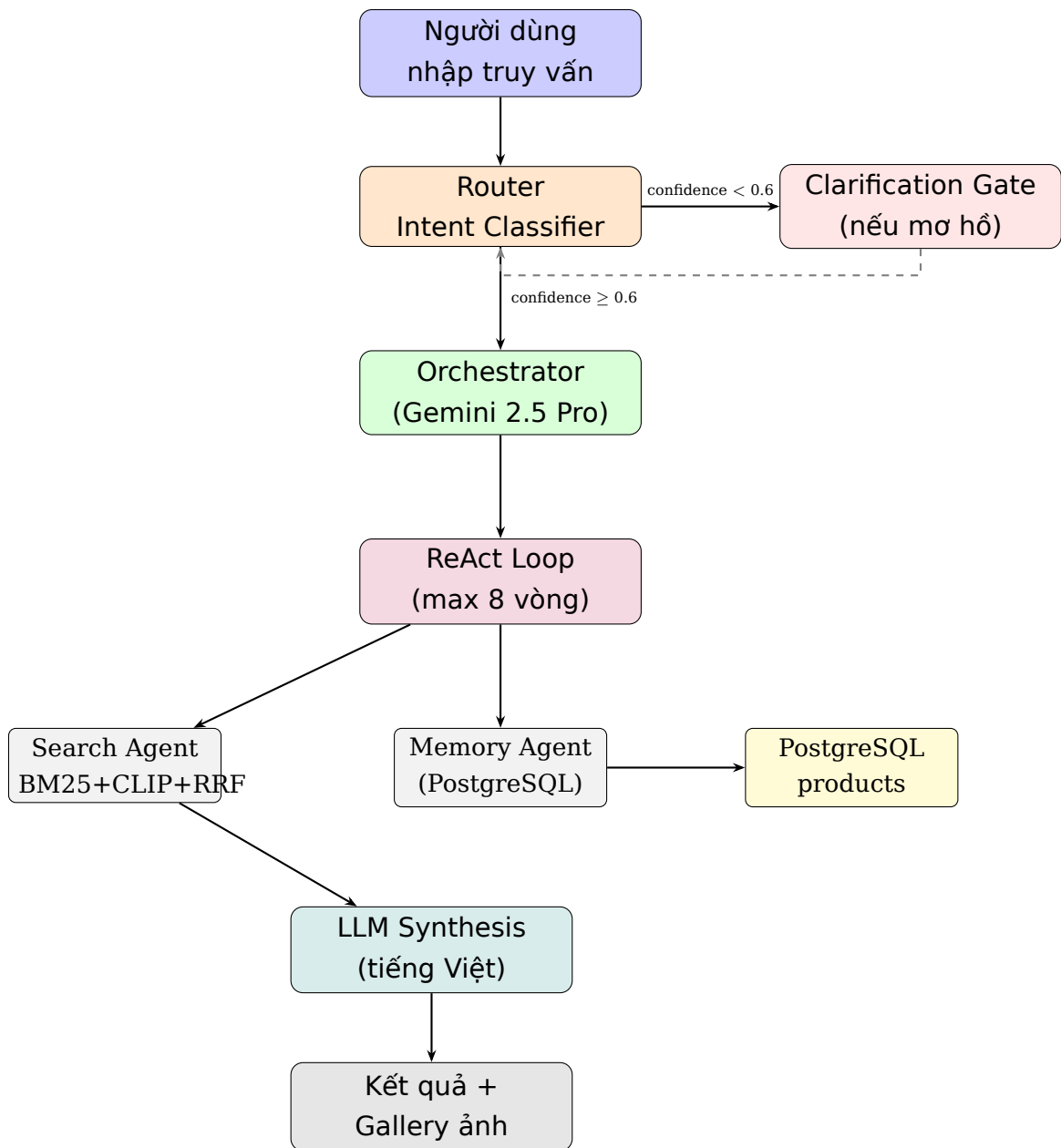
- `load_products`: Thay thế `pd.read_csv()`, đọc trực tiếp từ PostgreSQL.
- `ingest_from_csv`: Hàm chạy một lần để di chuyển dữ liệu, sử dụng `ON CONFLICT DO NOTHING` đảm bảo idempotent.
- `execute_values`: Batch insert hiệu năng cao (200 hàng/lần).
- `JSONB`: Lưu danh sách liked items và query history linh hoạt không cần thêm bảng phụ.

Chương 5

Agent v2.0 — PATH 1: Text-to-Image Search

5.1 Kiến trúc tổng thể Agent

Agent v2.0 được xây dựng theo mô hình **Orchestrator + Sub-agents + ReAct Loop**. Thay vì chạy pipeline cố định, Gemini 2.5 Pro đóng vai trò Orchestrator: phân tích yêu cầu, lập kế hoạch, gọi các tool phù hợp, và tổng hợp kết quả.



5.2 Bước 1 — Phân loại Intent (Router)

Phương pháp: LLM Few-shot Intent Classification

Gemini phân loại truy vấn thành 5 nhóm intent với điểm tin cậy (*confidence score*). Nếu $\text{confidence} < 0.6$, agent sẽ chuyển sang bước hỏi làm rõ thay vì tìm kiếm ngay.

Intent	Điều kiện	Hành động
text_search	Tìm sản phẩm cụ thể	Tìm kiếm ngay
outfit_request	Yêu cầu gợi ý trang phục	Tìm + phối đồ
follow_up	Tham chiếu lượt trước	Dùng ngữ cảnh session
out_of_scope	Không liên quan thời trang	Từ chối lịch sự
unclear	Thông tin quá mơ hồ	Hỏi làm rõ

```

1 import re, json
2 from dataclasses import dataclass, field
3 from enum import Enum
4 from typing import Dict, List
5
6 class IntentType(str, Enum):
7     TEXT_SEARCH = "text_search"
8     OUTFIT_REQUEST = "outfit_request"
9     FOLLOW_UP = "follow_up"
10    OUT_OF_SCOPE = "out_of_scope"
11    UNCLEAR = "unclear"
12
13 @dataclass
14 class IntentResult:
15     intent: IntentType
16     confidence: float
17     extracted_filters: Dict = field(default_factory=dict)
18     clarification_question: str = ""
19
20 CONFIDENCE_THRESHOLD = 0.6
21
22 def classify_intent(query: str,
23                   conversation_history: List[Dict],
24                   llm) -> IntentResult:
25     """
26     Phan loai intent bang LLM few-shot.
27     Tra ve IntentResult voi confidence score va filters trích xuất.
28     """
29     history_str = ""
30     if conversation_history:
31         history_str = "Lịch sử hội thoại:\n" + "\n".join(
32             f" {t['role']}: {t['content']}"
33             for t in conversation_history[-4:] # 4 lượt gần nhất
34         )
35
36     prompt = f"""Ban là bộ phận phân loại intent cho hệ thống thời trang.
37 {history_str}
38
39 Phan loai truy vấn vào 1 trong 5 nhóm:

```

```

40 - text_search: tim san pham cu the ("ao so mi xanh", "vay do di tiec")
41 - outfit_request: goi y trang phục cho dịp cu the
42 - follow_up: tham chieu den ket qua truoc ("con mau den khong?")
43 - out_of_scope: khong lien quan thoi trang
44 - unclear: qua mo ho, thieu thong tin
45
46 Tra ve ONLY JSON, khong giai thich:
47 {{
48   "intent": "<ten intent>",
49   "confidence": <so thap phan 0.0-1.0>,
50   "filters": {{
51     "category": "<T-Shirt|Dress|Pants|Shoes|Jacket|...> hoac null",
52     "color": "<ten mau> hoac null",
53     "occasion": "<office|party|casual|sport|...> hoac null",
54     "style": "<phong cach> hoac null"
55   }},
56   "clarification_question": "<cau hoi neu chua ro, de trong neu du>"
57 }}
58
59 Query: "{query}"
60 ""
61
62 try:
63     response = llm.complete(prompt).text
64     json_str = re.search(r'\{.*\}', response, re.DOTALL).group()
65     data = json.loads(json_str)
66     filters = {k: v for k, v in
67                 data.get("filters", {}).items() if v is not None}
68     return IntentResult(
69         intent=IntentType(data.get("intent", "unclear")),
70         confidence=float(data.get("confidence", 0.5)),
71         extracted_filters=filters,
72         clarification_question=data.get(
73             "clarification_question", "")
74     )
75 except Exception as e:
76     # Fallback an toan neu LLM tra ve sai dinh dang
77     return IntentResult(
78         intent=IntentType.UNCLEAR, confidence=0.3,
79         clarification_question=(
80             "Ban co the mo ta chi tiet hon "
81             "san pham ban muon tim khong?")
82     )

```

Listing 5.1: Intent Classification — LLM few-shot với confidence score

Giải thích tác vụ:

- **Lịch sử hội thoại** (4 lượt gần nhất): Giúp phân loại chính xác hơn với follow-up query.
- **Trích xuất filters**: LLM tự động rút ra category, color, occasion ngay trong bước classification — tiết kiệm một lần gọi LLM.
- **Fallback**: Bất kỳ lỗi parse JSON nào đều được xử lý an toàn, trả về intent unclear để trigger clarification.

5.3 Bước 2 — Clarification Gate

Phương pháp: Proactive Clarification

Khi confidence < 0.6, Agent **không đoán** mà hỏi lại người dùng. Cơ chế này tránh tìm kiếm sai, lãng phí tài nguyên và trả về kết quả không liên quan.

Truy vấn mơ hồ	Câu hỏi làm rõ của Agent
"tôi muốn áo"	"Bạn muốn áo sơ mi, áo thun hay áo khoác? Dịp sử dụng là gì?"
"cái gì đẹp"	"Bạn đang tìm sản phẩm cho dịp nào? Công sở, dạo phố hay sự kiện đặc biệt?"
"đồ đi tiệc"	"Tiệc trong nhà hay ngoài trời? Bạn thích màu sắc gì?"
"phong cách này"	"Bạn có thể mô tả cụ thể hơn không? Ví dụ: minimalist, streetwear, formal?"

```

1 def process(self, user_input: str, session_id: str,
2     conversation_history: List[Dict] = None) -> Dict:
3     """
4     Diem vào chính của Agent cho mọi lượt hỏi thoại.
5     Tra ve: {answer, image_paths, reasoning_trace,
6         needs_clarification, clarification_question}
7     """
8     history = conversation_history or []
9
10    # -- Buoc 1: Phan loai Intent --
11    intent_result = classify_intent(user_input, history, self.llm)
12
13    # -- Clarification Gate --

```

```

14     if (intent_result.confidence < CONFIDENCE_THRESHOLD
15         or intent_result.intent == IntentType.UNCLEAR):
16         cq = (intent_result.clarification_question or
17             "Ban co the mo ta chi tiet hon san pham "
18             "ban muon tim khong?")
19         return {
20             "answer": cq,
21             "image_paths": [],
22             "reasoning_trace": [
23                 f"Intent chua ro (confidence="
24                 f"{intent_result.confidence:.2f}) -> hoi lai"],
25             "needs_clarification": True,
26             "clarification_question": cq,
27             "result_ids": []
28         }
29
30     # -- Out of scope Gate --
31     if intent_result.intent == IntentType.OUT_OF_SCOPE:
32         return {
33             "answer": (
34                 "Xin loi, toi chi ho tro tim kiem va tu van thoi "
35                 "trang. Ban co cau hoi nao lien quan den quan ao, "
36                 "giay dep, phu kien khong?"),
37             "image_paths": [],
38             "reasoning_trace": ["Out of scope -> tu choi lich su"],
39             "needs_clarification": False,
40             "clarification_question": "",
41             "result_ids": []
42         }
43
44     # -- Tiep tuc xu ly voi intent ro rang --
45     # ...

```

Listing 5.2: Luồng Clarification Gate trong Agent.process()

5.4 Bước 3 — Memory Agent (PostgreSQL)

```

1 from collections import Counter
2
3 class MemoryAgent:
4     """
5     Luu tru va truy xuat lich su/so thich nguoi dung trong PostgreSQL.
6     Thay the in-memory dict bang persistent storage.
7     """
8
9     def __init__(self, db: FashionProductDB):

```

```

10     self.db = db
11     self._cache: Dict[str, Dict] = {} # cache cục bộ trong session
12
13     def get_preferences(self, session_id: str) -> Dict:
14         """Trích xuất sở thích tích lũy từ lịch sử liked items."""
15         session = self._get_or_load(session_id)
16         liked = session.get("liked_items", [])
17         if not liked:
18             return {}
19
20         cats = Counter(i["label"] for i in liked if i.get("label"))
21         cols = Counter(i["color"] for i in liked if i.get("color"))
22         prefs = {}
23         if cats: prefs["preferred_category"] = cats.most_common(1)[0][0]
24         if cols: prefs["preferred_color"] = cols.most_common(1)[0][0]
25         return prefs
26
27     def log_query(self, session_id: str, query: str,
28                  intent: str, result_count: int):
29         """Ghi lại lịch sử truy vấn vào PostgreSQL."""
30         import pandas as pd
31         session = self._get_or_load(session_id)
32         history = session.get("query_history", [])
33         history.append({
34             "query": query, "intent": intent,
35             "result_count": result_count,
36             "ts": pd.Timestamp.now().isoformat()
37         })
38         session["query_history"] = history[-20:] # giữ 20 lượt gần nhất
39         self._save(session_id, session)
40
41     def add_liked_item(self, session_id: str, image_id: str,
42                      label: str, color: str):
43         """Lưu sản phẩm người dùng đã xem/thích."""
44         session = self._get_or_load(session_id)
45         liked = session.get("liked_items", [])
46         liked.append({"image_id": image_id,
47                     "label": label, "color": color})
48         session["liked_items"] = liked[-50:] # giữ 50 sản phẩm
49         self._save(session_id, session)
50
51     def _get_or_load(self, session_id: str) -> Dict:
52         if session_id not in self._cache:
53             self._cache[session_id] = self.db.get_session(session_id)
54         return self._cache[session_id]
55
56     def _save(self, session_id: str, data: Dict):

```

```

57     self._cache[session_id] = data
58     self.db.upsert_session(session_id, data) # ghi PostgreSQL

```

Listing 5.3: Memory Agent — lưu/đọc sở thích từ PostgreSQL

Giải thích tác vụ:

- **Cache cục bộ:** Tránh đọc PostgreSQL nhiều lần trong cùng một request.
- **Sở thích tích lũy:** Tự động tổng hợp category và màu yêu thích từ lịch sử, dùng để làm giàu truy vấn tìm kiếm.
- **JSONB:** Lưu danh sách linh hoạt mà không cần thêm bảng, phù hợp với schema dynamique của session data.

5.5 Bước 4 — Vòng lặp ReAct

Phương pháp: ReAct (Reason + Act) Loop

ReAct là vòng lặp trung tâm: **Thought** → **Action** → **Observation** → (lặp lại nếu cần, tối đa 8 vòng). Gemini vừa lý luận vừa chủ động chọn tool phù hợp dựa trên quan sát từng bước.

```

1  MAX_REACT_LOOPS = 8
2
3  class Path1Agent:
4      """Agent chính cho PATH 1: Text-to-Image Fashion Search."""
5
6      def __init__(self, llm, retriever, memory: MemoryAgent,
7                  feedback, db: FashionProductDB):
8          self.llm = llm
9          self.retriever = retriever
10         self.memory = memory
11         self.feedback = feedback
12         self.db = db
13
14         # — THOUGHT: Orchestrator lập kế hoạch —————
15         def _plan(self, state: 'AgentState') -> List[Dict]:
16             """
17             Gemini tạo kế hoạch dưới dạng JSON array các tool calls.
18             Ví dụ: [{"tool": "search", "query": "...", "filters": {}}]
19             """
20             prompt = f"""Ban là orchestrator cho hệ thống tìm kiếm thời trang.
21

```

```

22 Query: "{state.current_query}"
23 Intent: {state.intent}
24 Filters trich xuất: {json.dumps(state.filters)}
25 So thích nguoi dung: {json.dumps(state.user_preferences)}
26
27 Tao ke hoach toi gian (1-3 buoc). Tool kha dung:
28 - search: {"tool": "search", "query": "...", "filters": {}}
29 - memory_enrich: {"tool": "memory_enrich"}
30   (lam giau query bang so thich, chi dung khi co preferences)
31 - outfit_hints: {"tool": "outfit_hints", "occasion": "..."}
32   (chi cho outfit_request intent)
33
34 Tra ve ONLY JSON array: ""
35
36     try:
37         resp = self.llm.complete(prompt).text
38         arr = re.search(r'\[.*\]', resp, re.DOTALL).group()
39         plan = json.loads(arr)
40         state.reasoning_trace.append(
41             f"Ke hoach: {json.dumps(plan)}")
42         return plan
43     except:
44         # Fallback: tim kiem truc tiep
45         return [{"tool": "search",
46                 "query": state.current_query,
47                 "filters": state.filters}]
48
49 # — ACTION: Thuc thi tool —————
50 def _execute_tool(self, tool_call: Dict,
51                   state: 'AgentState') -> List:
52     tool = tool_call.get("tool", "search")
53     state.tool_calls.append(tool)
54
55     if tool == "search":
56         query = tool_call.get("query", state.current_query)
57         filters = tool_call.get("filters", state.filters)
58         state.reasoning_trace.append(
59             f"search('{query}', filters={filters})")
60         return search_fashion(query, self.retriever, filters)
61
62     elif tool == "memory_enrich":
63         # Lam giau query bang so thich tu PostgreSQL session
64         prefs = state.user_preferences
65         enriched = state.current_query
66         if prefs.get("preferred_color"):
67             enriched += f" {prefs['preferred_color']}"
68         if prefs.get("preferred_category"):

```

```

69         enriched += f" {prefs['preferred_category']}"
70     state.reasoning_trace.append(
71         f"memory_enrich -> '{enriched}'")
72     return search_fashion(enriched, self.retriever,
73                           state.filters)
74
75     elif tool == "outfit_hints":
76         # Quy tắc outfit theo dịp
77         occasion = tool_call.get("occasion", "casual")
78         OCCASION_MAP = {
79             "office" : ["Shirt", "Pants", "Shoes"],
80             "party" : ["Dress", "Heels"],
81             "casual" : ["T-Shirt", "Jeans", "Sneakers"],
82             "interview" : ["Shirt", "Pants", "Shoes"],
83         }
84         categories = OCCASION_MAP.get(occasion.lower(),
85                                       ["Shirt", "Pants"])
86         results = []
87         for cat in categories:
88             q = f"{cat} {state.filters.get('color', '')} {occasion}"
89             r = search_fashion(q.strip(), self.retriever,
90                               {"category": cat})
91             results.extend(r[:2])
92         return deduplicate_nodes(results)
93
94     return []
95
96     # — OBSERVATION + Fallback —————
97     def _broaden_search(self, state: 'AgentState') -> List:
98         """Neu khong co ket qua, ha nguoi relevance."""
99         original = self.retriever.relevance_threshold
100         self.retriever.relevance_threshold = max(
101             0.0, original - 0.2)
102         results = search_fashion(state.current_query,
103                                 self.retriever, {})
104         self.retriever.relevance_threshold = original
105         state.reasoning_trace.append(
106             "Fallback: broadened search (ha nguoi relevance)")
107         return results

```

Listing 5.4: ReAct Loop — Orchestrator Planning và Tool Execution

5.6 Bước 5 — LLM Synthesis và Memory Update

```

1     def _synthesize(self, state: 'AgentState') -> str:
2         """

```



```

3         Gemini tong hop ket qua timkiem thanh cau tra loi
4         bang tieng Viet, kem ly do lua chon.
5         """
6         if not state.retrieved_nodes:
7             return ("Rat tiec, khong tim thay san pham phu hop. "
8                     "Ban co muon thu mo ta khac khong?")
9
10        # Tao context tu cac node tim duoc
11        parts = []
12        for i, node in enumerate(state.retrieved_nodes[:6]):
13            actual = (node.node if hasattr(node, 'node') else node)
14            meta = actual.metadata
15            parts.append(
16                f"[SP{i+1}] {meta.get('category','?')} | "
17                f"Mau: {meta.get('color','?')} | "
18                f"Mo ta: {meta.get('full_description','')[:120]}")
19        context = "\n".join(parts)
20
21        pref_hint = ""
22        if state.user_preferences:
23            pref_hint = (f"\nSo thích người dùng: "
24                        f"{json.dumps(state.user_preferences)}")
25
26        prompt = f"""Ban la tu van vien thoi trang chuyen nghiep.
27 {pref_hint}
28
29 Cau hoi: "{state.current_query}" (Intent: {state.intent})
30
31 San pham tim duoc:
32 {context}
33
34 Hay:
35 1. Gioi thieu ngan gon san pham phu hop nhat
36 2. Giai thích ly do lua chon (mau, kieu dang, dip dung)
37 3. Neu la outfit request: goi y cach phoi do
38 4. De xuat ngan 1-2 buoc tiep theo cho nguoi dung
39
40 Tra loi bang tieng Viet, chan thuc, tu van nhu chuyen gia."""
41
42        try:
43            return self.llm.complete(prompt).text
44        except Exception as e:
45            return (f"Tim thay {len(state.retrieved_nodes)} "
46                    f"san pham. Loi tong hop: {e}")
47
48    def process(self, user_input: str, session_id: str,
49               conversation_history=None) -> Dict:

```

```

50     # ... [intent, clarify, plan đã trình bày ở trên] ...
51
52     # — ReAct Loop —————
53     all_results = []
54     for step in plan:
55         if state.react_loop_count >= MAX_REACT_LOOPS:
56             break
57         state.react_loop_count += 1
58         results = self._execute_tool(step, state)
59         all_results.extend(results)
60         state.reasoning_trace.append(
61             f"Observation: {len(results)} kết quả "
62             f"từ '{step.get('tool')}'")
63
64     state.retrieved_nodes = deduplicate_nodes(all_results)[:10]
65
66     # Fallback nếu rỗng
67     if not state.retrieved_nodes:
68         state.retrieved_nodes = self._broaden_search(state)[:6]
69
70     # — Tổng hợp và cập nhật PostgreSQL —————
71     state.final_answer = self._synthesize(state)
72     self.memory.log_query(session_id, user_input,
73                           state.intent.value,
74                           len(state.retrieved_nodes))
75     return {
76         "answer"           : state.final_answer,
77         "image_paths"      : self._extract_image_paths(state),
78         "reasoning_trace"  : state.reasoning_trace,
79         "needs_clarification": False,
80         "result_ids"       : self._extract_result_ids(state)
81     }

```

Listing 5.5: LLM Synthesis và cập nhật PostgreSQL sau mỗi lượt

Chương 6

Ví dụ hoạt động end-to-end

6.1 Kịch bản: Tìm trang phục đi họp công ty

Ví dụ: “Tôi muốn tìm một trang phục để đi họp trong công ty”

Đây là truy vấn điển hình với **intent đủ rõ** (outfit request, dịp công ty) nhưng **thiếu một số chi tiết** (màu sắc, kiểu loại cụ thể). Agent sẽ hỏi thêm trong lần 1, rồi tìm kiếm hoàn chỉnh ở lần 2.

6.1.1 Lượt 1 — Truy vấn ban đầu

Diễn biến Lượt 1

Người dùng: “Tôi muốn tìm một trang phục để đi họp trong công ty”

[Step 1 — Intent Classification]

- Intent phân loại: `outfit_request`
- Confidence: **0.72** $\geq 0.6 \rightarrow$ đủ để tiếp tục
- Filters trích xuất: `{occasion: "office"}`
- Category/color: `null` (chưa đủ thông tin chi tiết)

[Step 2 — Memory Lookup (PostgreSQL)]

- Session mới $\rightarrow$ không có sở thích lưu trữ

[Step 3 — Orchestrator Planning]

Gemini tạo kế hoạch:

```
[  
  {"tool": "outfit_hints", "occasion": "office"},
```

```
{ "tool": "search", "query": "formal shirt office meeting",  
  "filters": { "occasion": "office" }  
}
```

[ReAct Loop — Vòng 1]

Action: outfit_hints(occasion="office")

Observation: categories = [Shirt, Pants, Shoes]

Tìm kiếm từng category với query “Shirt office meeting”, “Pants office”, “Shoes formal”

[ReAct Loop — Vòng 2]

Action: search("formal shirt office meeting", {occasion: office})

Observation: 8 kết quả (5 TextNode + 3 ImageNode) sau RRF + reranker

[LLM Synthesis — Gemini]

Agent trả lời:

“Cho buổi họp công ty, tôi gợi ý bộ trang phục smart-casual: (1) Áo sơ mi cotton dệt thoi với cổ điểm nút, kiểu dáng slim-fit — thanh lịch, phù hợp môi trường chuyên nghiệp; (2) Quần tây vải chinos màu ghi hoặc navy; (3) Giày da kín mũi hoặc loafer sạch sẽ.

Bạn muốn tôi tìm thêm theo màu sắc cụ thể không, ví dụ navy blue hoặc trắng?”

6.1.2 Lượt 2 — Follow-up với thông tin bổ sung

Diễn biến Lượt 2

Người dùng: “Cho tôi áo sơ mi màu trắng, quần navy”

[Step 1 — Intent Classification]

- Intent: follow_up (tham chiếu kết quả lần 1)
- Confidence: **0.88**
- Filters: {category: "Shirt", color: "White"} và {category: "Pants", color: "Navy"}

[Step 2 — Memory Agent]

- Tải ngữ cảnh session: dịp *office*, đã xem áo sơ mi + quần tây

[Orchestrator Plan]

```
[  
  { "tool": "search", "query": "white cotton office shirt",
```

```

    "filters": {"category": "Shirt", "color": "White"}},
    {"tool": "search", "query": "navy blue office pants",
     "filters": {"category": "Pants", "color": "Navy Blue"}}
  ]

```

[ReAct Loop — 2 vòng tìm kiếm]

BM25 khớp chính xác: “Shirt. White.” → rank cao nhất

FashionCLIP hiểu ngữ nghĩa: “slim fit woven cotton formal shirt” → bổ sung kết quả semantic

[Kết quả]

Agent trả lời + gallery 6 ảnh:

“Đây là gợi ý phối đồ hoàn chỉnh cho buổi họp: (1) Áo sơ mi trắng cotton slim-fit với cổ bẻ — [SP1, SP2, SP3] (2) Quần tây navy blue kiểu chinos — [SP4, SP5]

Combo trắng + navy là classic combination, rất phù hợp cho môi trường tech/corporate. Bạn có muốn tôi gợi ý thêm giày phối không?”

[Memory Update — PostgreSQL]

Lưu vào user_sessions: liked_items += [{SP1, Shirt, White}, {SP4, Pants, Navy}]

6.1.3 Kịch bản từ chối — Sản phẩm không có trong database

Xử lý khi không tìm thấy kết quả

Người dùng: “Tìm áo Gucci cho tôi”

[Intent]: text_search, confidence 0.91

[Filters]: {brand: "Gucci"} — nhưng hệ thống không lưu brand

[ReAct Loop]: Tìm kiếm với query “Gucci luxury fashion” → 0 kết quả có relevance ≥ 0.3

[Fallback]: Giảm threshold 0.3 → 0.1 → vẫn không đủ tốt

Agent từ chối lịch sự:

“Rất tiếc, bộ sưu tập hiện tại chưa bao gồm sản phẩm thương hiệu Gucci. Tuy nhiên, tôi có thể tìm các sản phẩm có phong cách tương tự (luxury, minimalist, formal). Bạn có muốn thử không?”

Chương 7

Giao diện và triển khai

7.1 Giao diện Gradio

```
1 import gradio as gr
2 import uuid
3
4 _sessions: Dict[str, Dict] = {}
5
6 def chat_handler(message: str, history: List,
7                  session_state: Dict):
8     """Hàm xử lý từng lượt chat trong Gradio."""
9     if not message.strip():
10         return "", history, session_state
11
12     # Khởi tạo session
13     if "session_id" not in session_state:
14         session_state["session_id"] = str(uuid.uuid4())
15         session_state["conv_history"] = []
16
17     sid = session_state["session_id"]
18     conv_h = session_state["conv_history"]
19
20     result = agent.process(message, sid, conv_h)
21
22     # Cập nhật lịch sử hội thoại
23     conv_h.append({"role": "user", "content": message})
24     conv_h.append({"role": "assistant", "content": result["answer"]})
25     session_state["conv_history"] = conv_h[-20:]
26
27     # Định dạng trả lời: thêm reasoning trace an (collapsible)
28     answer = result["answer"]
29     if result.get("reasoning_trace"):
```

```

30     trace = "\n".join(result["reasoning_trace"])
31     answer += (f"\n\n<details><summary>Reasoning trace"
32               f"</summary>\n\n```\n{trace}\n```\n</details>")
33
34     history.append((message, answer))
35     return "", history, session_state
36
37
38 with gr.Blocks(theme=gr.themes.Soft(),
39               title="Fashion Agent v2.0") as demo:
40     gr.Markdown("""
41     ## Fashion Agent v2.0 – PATH 1: Text-to-Image Search
42     > Powered by FashionCLIP 2.0 + BM25 + Gemini 2.5 Pro + PostgreSQL
43     """)
44
45     session_state = gr.State({})
46     chatbot = gr.Chatbot(label="Fashion Assistant", height=500)
47
48     with gr.Row():
49         msg = gr.Textbox(
50             placeholder="Ví dụ: Tìm áo sơ mi xanh cho buổi họp...",
51             label="Câu hỏi của bạn", scale=5)
52         send = gr.Button("Gửi", variant="primary", scale=1)
53
54     gr.Examples(inputs=msg, examples=[
55         ["Tìm trang phục đi họp công ty"],
56         ["Áo sơ mi màu trắng, quần navy"],
57         ["Gợi ý outfit cho bữa tiệc cuối"],
58         ["red dress suitable for party"],
59     ])
60
61     send.click(chat_handler,
62               [msg, chatbot, session_state],
63               [msg, chatbot, session_state])
64     msg.submit(chat_handler,
65               [msg, chatbot, session_state],
66               [msg, chatbot, session_state])
67
68     demo.launch(share=True, debug=False)

```

Listing 7.1: Gradio Chat Interface tích hợp Path1Agent

7.2 Cấu hình triển khai trên server

```

1 # requirements.txt
2 torch torchvision

```

```
3 transformers accelerate
4 llama-index-core
5 llama-index-vector-stores-qdrant qdrant_client
6 llama-index-llms-google-genai google-generativeai
7 llama-index-retrievers-bm25 rank_bm25
8 sentence-transformers
9 psycopg2-binary          # PostgreSQL driver
10 pandas pillow rapidfuzz
11 gradio fastapi uvicorn
12
13 # Bien moi truong (.env)
14 DATABASE_URL=postgresql://user:password@db-host:5432/fashion_db
15 GOOGLE_API_KEY=your_gemini_api_key
16 QDRANT_PATH=/app/data/qdrant_db
17 MODEL_CACHE=/app/data/models
18
19 # Khoi dong (Docker / Cloud Run)
20 # uvicorn main:app --host 0.0.0.0 --port 8080
```

Listing 7.2: Cấu hình môi trường và khởi động hệ thống

Chương 8

Đánh giá và Kết luận

8.1 Chỉ số đánh giá (theo kế hoạch v2.0)

Metric	Ý nghĩa	RAG v1.0	Agent v2.0
Recall@5 (PATH 1)	Top-5 đúng category/color	$\geq 85\%$	$\geq 88\%$
MRR	Vị trí kết quả đúng đầu tiên	≥ 0.75	≥ 0.80
Hit Rate@5	≥ 1 kết quả đúng trong top-5	$\geq 90\%$	$\geq 93\%$
Task Completion Rate	Không crash, có kết quả	N/A	$\geq 95\%$
Faithfulness (RAGAS)	Không bịa thông tin	N/A	≥ 0.87
Agent Loop Latency P95	Tổng thời gian ReAct loop	$< 15s$	$< 15s$

8.2 Điểm mạnh thiết kế

1. **Intent-first:** Agent xác định intent trước khi hành động → tránh tìm kiếm sai.
2. **Clarification gate:** Với query mơ hồ, hỏi thêm thay vì đoán → chất lượng cao hơn.
3. **Graceful degradation:** Từ chối lịch sự khi không có sản phẩm, kèm gợi ý thay thế.
4. **Hybrid retrieval:** BM25 + FashionCLIP + RRF tận dụng điểm mạnh của cả hai.
5. **PostgreSQL:** Persistent, scalable, hỗ trợ concurrent access tốt hơn CSV.

6. **Semantic token (caption):** LLM sinh mô tả phong phú → FashionCLIP hiểu sâu hơn nhãn ngắn.

8.3 Rủi ro và hướng giải quyết

Rủi ro	Mức độ	Giải pháp
Latency cao (nhiều Gemini API calls)	Cao	Parallel tool calls, cache query expansion
Hallucination LLM	Cao	RAGAS faithfulness ≥ 0.87 , template fallback
BM25 rebuild khi khởi động	Thấp	Phase 3: serialize BM25 index ra disk
Cold start PostgreSQL	Thấp	Connection pooling (psycopg2.pool)

8.4 Kết luận

Đề tài đã xây dựng thành công hệ thống tìm kiếm thời trang đa phương thức theo lộ trình hai giai đoạn rõ ràng. Giai đoạn RAG v1.0 thiết lập nền tảng tìm kiếm hybrid với FashionCLIP 2.0 và BM25, trong khi Agent v2.0 PATH 1 nâng cấp lên kiến trúc chủ động với vòng lặp ReAct, phân loại intent, clarification gate, và persistent memory qua PostgreSQL.

Việc chuyển đổi từ CSV sang PostgreSQL không chỉ giải quyết vấn đề triển khai server mà còn mở ra khả năng mở rộng quy mô, hỗ trợ nhiều người dùng đồng thời, và lưu trữ lịch sử/sở thích để cá nhân hóa kết quả tìm kiếm về lâu dài.

Hướng phát triển tiếp theo:

- **Phase 3:** Mở rộng dataset lên 15K-100K sản phẩm, fine-tune reranker, Redis session.
- **Phase 4:** Virtual Try-on (IDM-VTON), fine-tune FashionCLIP trên dữ liệu riêng, A/B testing.
- **PATH 2:** Tìm kiếm bằng hình ảnh (Image-to-Image) với Composed Search.